Activation & Gradient Histograms, Worked Out by Hand

Reading distribution shape as a training X-ray

A complete numerical walk-through of how three numbers — *mean*, *standard deviation*, and a *dead*-or-*saturated* fraction — plus a tiny bin-count histogram turn a layer's raw activations into a one-glance health report. We take a 12-element ReLU layer that is quietly dying, compute every statistic by hand, build its histogram bin by bin, and then contrast it with a healthy normalized layer. Nothing is hand-waved — every number below was produced by the NumPy reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It fixes one small layer of ReLU outputs, $a = [0, 0, 1.2, 0, 3.1, 0, 0, 2.0, 0, 0.5, 0, 4.0]$, and computes the exact summaries the course prints in its `ActivationMonitor.summary()`: `mean`, `std`, and `dead_ratio`. It then bins those 12 numbers into a histogram you can draw on paper, and repeats the exercise for a healthy layer and for a saturating sigmoid/tanh pre-activation, so the *shape* of each distribution is unmistakable.

How to read this. Each statistic is derived once, slowly, with a *Mini-example* box isolating the arithmetic, then read back as a diagnostic. The goal is not the formula — you know how to average twelve numbers — but the *judgement*: which value of the dead fraction means "stop training and lower the learning rate," and why you must inspect this *per layer*, never aggregated.

Concepts covered, in order: the three layer summaries (mean, std, dead / saturated fraction) · population vs. sample std and why it barely matters here · the dead-ReLU fraction 7/12 and what it signals · building a histogram as bin counts · a healthy zero-mean unit-scale layer for contrast · saturation of tanh/sigmoid via $|z| > 4$ and the vanishing-gradient link · reading all of this as a training X-ray, before the loss curve moves.

1 The setup: one layer, twelve activations

A real hidden layer emits thousands of activations per forward pass — far too many to eyeball. The course's fix is to collapse each layer's output tensor to a tiny *summary* (`mean`, `std`, `dead_ratio`) and an optional *histogram*, then scan those across layers. To follow the machinery on paper we shrink one ReLU layer to $n = 12$ outputs, small enough to add by hand but large enough to be visibly sick:

$$a = [0,\ 0,\ 1.2,\ 0,\ 3.1,\ 0,\ 0,\ 2.0,\ 0,\ 0.5,\ 0,\ 4.0].$$

Seven of the twelve entries are exactly 0 — the fingerprint of a ReLU whose inputs have drifted negative. The five survivors are 1.2, 3.1, 2.0, 0.5, 4.0.

Summary	Symbol	What it answers
Mean	μ	where is the distribution centred?
Std	σ	how wide / how much scale?
Dead fraction	f_{dead}	share of units stuck at exactly 0 (ReLU)
Saturated fraction	f_{sat}	share with $\lvert z\rvert > 4$ (sigmoid / tanh)
Histogram	counts per bin	the full *shape*, not just two moments

Intuition

The *shape* of a layer's activation distribution is your training X-ray. A collapse toward 0 (a tall spike at zero, an empty right tail) or a blow-up (an ever-widening spread, mass piling at the rails) is visible in the histogram *immediately* — often several hundred steps before the loss curve bends or goes `NaN`. Two scalars (μ, σ) compress it; the histogram preserves it. Read both.

2 Step 1 — The mean: where the mass sits

The mean is the plain average of the twelve outputs. Only the five non-zero entries contribute to the sum:

$$\mu = \frac{1}{n}\sum_{i=1}^{n} a_i = \frac{0{+}0{+}1.2{+}0{+}3.1{+}0{+}0{+}2.0{+}0{+}0.5{+}0{+}4.0}{12} = \frac{10.8}{12} = \boxed{0.9}.$$

A mean of 0.9 for a ReLU layer is unremarkable on its own — ReLU outputs are non-negative, so a positive mean is expected. The mean alone hides the illness; it is dragged up by the few large survivors (3.1, 4.0) even as most units are dead.

Watch out

The mean is a poor dead-detector. Here seven of twelve units are dead, yet $\mu = 0.9 > 0$ looks healthy. A handful of large activations can mask a mostly-dead layer. This is exactly why the course logs `dead_ratio` *separately* from the mean — and why you should never declare a layer fine from its mean alone.

3 Step 2 — The standard deviation: how wide

Std measures spread around the mean. Form each deviation $a_i - \mu$, square it, and sum:

a_i	0	0	1.2	0	3.1	0	0	2.0	0	0.5	0	4.0
$a_i - \mu$	−0.9	−0.9	0.3	−0.9	2.2	−0.9	−0.9	1.1	−0.9	−0.4	−0.9	3.1
$(a_i - \mu)^2$	0.81	0.81	0.09	0.81	4.84	0.81	0.81	1.21	0.81	0.16	0.81	9.61

The seven dead units each contribute $(0-0.9)^2 = 0.81$; the survivors contribute the rest. Summing the bottom row:

$$\mathrm{SS} \;=\; \sum_i (a_i - \mu)^2 \;=\; 7(0.81) + 0.09 + 4.84 + 1.21 + 0.16 + 9.61 \;=\; 21.58.$$

Now there are two conventions, and it is worth seeing both because PyTorch's `tensor.std()` and NumPy's default disagree:

$$\sigma_{\text{pop}} \;=\; \sqrt{\tfrac{\mathrm{SS}}{n}} \;=\; \sqrt{\tfrac{21.58}{12}} \;=\; \sqrt{1.798} \;=\; \boxed{1.341}, \qquad \sigma_{\text{samp}} \;=\; \sqrt{\tfrac{\mathrm{SS}}{n-1}} \;=\; \sqrt{\tfrac{21.58}{11}} \;=\; \sqrt{1.962} \;=\; 1.401.$$

Mini-example — this operation in isolation

Population vs. sample std is the choice of denominator n vs. $n-1$. With SS = 21.58 and $n = 12$: dividing by 12 gives variance 1.798 and $\sigma = 1.341$; dividing by 11 gives 1.962 and $\sigma = 1.401$. The gap is $\sim 4.5\%$ here and shrinks like $\sqrt{n/(n-1)}$ as n grows. PyTorch's `.std()` uses $n-1$ by default; NumPy's `.std()` uses n. For a monitoring histogram the difference is noise — but report *which* you used so two dashboards agree.

A std of ≈ 1.34 next to a mean of 0.9 tells you the spread is comparable to the centre: the distribution is dominated by a spike at 0 with a thin, long right tail. That is precisely the silhouette of a dying ReLU.

4 Step 3 — The dead fraction: the decisive number

For a ReLU layer the diagnostic that actually flags trouble is the proportion of outputs that are *exactly* zero — units that, for this batch, pass no gradient and learn nothing:

$$f_{\text{dead}} \;=\; \frac{\#\{i : a_i = 0\}}{n}.$$

Counting the zeros in a — positions $1, 2, 4, 6, 7, 9, 11$ (one-indexed) — gives 7:

$$f_{\text{dead}} \;=\; \frac{7}{12} \;=\; \boxed{0.583}.$$

This is the value the course's monitor prints as `dead_ratio`. Nearly 60% of the layer is dark.

Watch out

All-zero (or mostly-zero) activations mean **dead ReLUs**: their pre-activations have gone negative, $\mathrm{ReLU}'(z) = 0$ there, so no gradient flows back through them and they are frozen. A high f_{dead} (rule of thumb: a sustained > 0.5, climbing across layers) is an action signal, not

a curiosity. Fixes, cheapest first: *lower the learning rate* (large updates are what kicked the units negative), switch to *LeakyReLU / GELU* (non-zero slope keeps a trickle of gradient), or use a *better initialisation* (Kaiming for ReLU). Inspect this *per layer* — averaging the dead fraction over the whole network hides the one sick layer that is poisoning the rest.

When this is the right choice

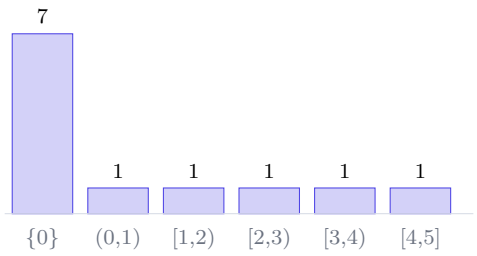
Reach for activation/gradient histograms exactly when the symptom is diffuse: the loss is *flat* or has gone `NaN`, or learning is *uneven across layers* (early layers move, late ones don't, or vice versa). Scalar logs like a single loss number cannot localise the fault; a per-layer histogram can point straight at the layer whose activations collapsed or exploded.

5 Step 4 — Building the histogram by hand

The two moments compress the layer to (μ, σ); the histogram keeps its full shape. Choose unit-width bins over the observed range $[0, 5)$ and tally how many of the twelve values fall in each. We split the first bin to expose the dead spike explicitly:

Bin	Count	Members
$\{0\}$ exactly	7	the seven dead units
$(0, 1)$	1	0.5
$[1, 2)$	1	1.2
$[2, 3)$	1	2.0
$[3, 4)$	1	3.1
$[4, 5]$	1	4.0
total	12	

Drawn as a bar chart, the counts are $7, 1, 1, 1, 1, 1$ — a single dominant spike at zero and a flat, sparse tail to the right:



You do not need the numbers $\mu = 0.9$ or $\sigma = 1.341$ to read this picture: the 58% spike at zero *is* the diagnosis. That is the entire point of plotting the shape rather than only logging two moments.

Intuition

A healthy ReLU histogram looks like a hill that starts at zero and falls off smoothly to the right — a modest pile near zero, a fat body, a thin tail. A *dying* ReLU histogram is a skyscraper at exactly zero with almost nothing beside it. The eye catches the difference between "hill" and "skyscraper" instantly; that pre-attentive read is why histograms beat staring at a column of means.

6 Step 5 — A healthy layer, for contrast

Compare a well-behaved layer of 12 post-normalization activations (roughly zero-mean, unit-scale, as you'd see after BatchNorm/LayerNorm):

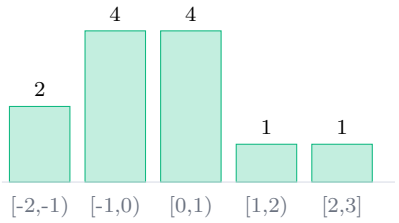
$$h = [-1.8,\ -0.9,\ -0.4,\ 0.1,\ -0.2,\ 1.5,\ 0.7,\ -1.1,\ 0.3,\ 2.0,\ -0.6,\ 0.4].$$

The summaries come out where you want them:

$$\mu_h = \frac{0.0}{12} = 0.000, \qquad \mathrm{SS}_h = 12.82, \qquad \sigma_{h,\mathrm{pop}} = \sqrt{\tfrac{12.82}{12}} = 1.034, \qquad f_{\mathrm{dead}} = 0, \quad f_{\mathrm{sat}} = 0.$$

Centred at zero, spread of order one, *no* dead units and *no* saturation. Its histogram (unit bins) is a tidy mound straddling zero:

bin	$[-2,-1)$	$[-1,0)$	$[0,1)$	$[1,2)$	$[2,3]$	total
count	2	4	4	1	1	12



The contrast is the whole lesson. Same number of values, same bin width, but the dead layer is a spike at one edge while the healthy layer is a roughly symmetric mound about zero. Healthy gradients behave the same way: small but *non-zero*, and of *similar scale across layers* — not vanishing in the deep layers nor exploding in the shallow ones.

7 Step 6 — Saturation: the sigmoid/tanh failure mode

ReLU dies at zero; the bounded activations (sigmoid, tanh) fail the opposite way — they *saturate*, pinning at their rails ± 1 where the gradient vanishes. The course's detector flags a pre-activation as saturated when $|z| > 4$. Take a layer of pre-activations:

$$z = [-6.0,\ -4.5,\ -0.3,\ 0.8,\ -1.2,\ 5.1,\ 4.2,\ -0.1,\ 3.0,\ -5.5,\ 0.5,\ 2.1].$$

The entries with $|z| > 4$ are -6.0, -4.5, 5.1, 4.2, -5.5 — five of them:

$$f_{\text{sat}} = \frac{\#\{i : |z_i| > 4\}}{n} = \frac{5}{12} = \boxed{0.417}.$$

Why $|z| > 4$ is the right threshold: it is where the derivative collapses. For tanh, the local gradient is $1 - \tanh^2(z)$. Compare an unsaturated unit with a saturated one:

$$z = 0.8: \quad 1 - \tanh^2(0.8) = 0.559, \qquad z = 5.1: \quad 1 - \tanh^2(5.1) = 0.000149.$$

The saturated unit passes a gradient roughly 3,760× smaller — effectively zero. (The sigmoid story is identical in spirit: its peak slope is only 0.25 at $z = 0$, e.g. $\sigma'(0.5) = 0.235$, and it decays just as fast toward the rails.)

Watch out

Two distribution pathologies, opposite shapes, one habit. **Dead ReLUs**: a spike at 0, fixed by lower lr / LeakyReLU / better init. **Saturation**: mass piled at ± 1 (or $|z| > 4$), starving gradients — fixed by ReLU/GELU or a normalization layer before the activation. And an *ever-growing activation std* across steps is a third alarm: rising spread signals instability heading toward `NaN`, usually cured by a smaller learning rate or gradient clipping. In every case, inspect **per layer**, never aggregated — an average over a healthy and a dead layer can look perfectly fine while one layer is on fire.

8 The three layers, at a glance

Layer	μ	σ_{pop}	f_{dead}	f_{sat}	Verdict
Dying ReLU (a)	0.900	1.341	0.583	—	spike at 0: lower lr / LeakyReLU
Healthy norm. (h)	0.000	1.034	0.000	0.000	zero-mean, unit-scale mound: fine
Saturating tanh (z)	—	—	—	0.417	mass at ±rails: ReLU/GELU or norm

These five columns are the entire forward-pass health check the course automates in `ActivationMonitor.summary()`. Print them per layer every few hundred steps; when f_{dead} or f_{sat} climbs, or σ marches upward, you have caught the failure on the X-ray before it ever reached the loss curve.

9 Equation cheat-sheet

Mean	$\mu = \frac{1}{n}\sum_i a_i$
Sum of squares	$\mathrm{SS} = \sum_i (a_i - \mu)^2$
Std (population, NumPy default)	$\sigma_{\mathrm{pop}} = \sqrt{\mathrm{SS}/n}$
Std (sample, Torch default)	$\sigma_{\mathrm{samp}} = \sqrt{\mathrm{SS}/(n-1)}$
Dead fraction (ReLU)	$f_{\mathrm{dead}} = \frac{\#\{a_i = 0\}}{n}$
Saturated fraction (sig/tanh)	$f_{\mathrm{sat}} = \frac{\#\{\lvert z_i\rvert > 4\}}{n}$
Histogram bin count	$c_b = \#\{i : a_i \in \mathrm{bin}_b\}, \quad \sum_b c_b = n$
tanh local gradient	$1 - \tanh^2(z) \quad (\to 0 \text{ as } \lvert z\rvert \to \infty)$
Sigmoid local gradient	$\sigma(z)\,(1 - \sigma(z)) \quad$ (peak 0.25 at z=0)

Appendix A Reference implementation

Every number above is reproducible from the following program. The first block recomputes the by-hand statistics and histogram; the second shows the course's forward/backward hooks collecting per-layer histograms on a real model.

```
import numpy as np

# --- the dying-ReLU layer ---
a = np.array([0,0,1.2,0,3.1,0,0,2.0,0,0.5,0,4.0])
print("mean      =", round(a.mean(), 3))           # 0.9
print("SS        =", round(((a-a.mean())**2).sum(), 3))  # 21.58
print("std_pop   =", round(a.std(ddof=0), 3))       # 1.341  (NumPy default)
print("std_samp  =", round(a.std(ddof=1), 3))       # 1.401  (Torch default)
print("dead_frac =", round((a == 0).mean(), 3))     # 0.583  = 7/12

# histogram bin counts (unit bins over [0,5])
counts, edges = np.histogram(a, bins=[0,1,2,3,4,5])
print("counts    =", counts, "exact zeros =", int((a == 0).sum()))
# counts = [8 1 1 1 1]; of the 8 in [0,1), 7 are exactly 0 and one is 0.5

# --- healthy normalized layer ---
h = np.array([-1.8,-0.9,-0.4,0.1,-0.2,1.5,0.7,-1.1,0.3,2.0,-0.6,0.4])
print("h: mean =", round(h.mean(),3), " std_pop =", round(h.std(ddof=0),3),
      " dead =", round((h==0).mean(),3))            # 0.0  1.034  0.0

# --- saturating tanh pre-activations ---
z = np.array([-6.0,-4.5,-0.3,0.8,-1.2,5.1,4.2,-0.1,3.0,-5.5,0.5,2.1])
print("sat_frac =", round((np.abs(z) > 4).mean(), 3))         # 0.417 = 5/12
print("tanh grad @0.8 =", round(1-np.tanh(0.8)**2, 3),
      " @5.1 =", round(1-np.tanh(5.1)**2, 6))        # 0.559  vs  0.000149
```

```
import torch, torch.nn as nn

class ActivationMonitor:
    """Forward + backward hooks -> per-layer activation/grad histograms."""
    def __init__(self, bins=20):
        self.bins = bins
        self.act, self.grad, self.hooks = {}, {}, []

    def register(self, model):
        for name, layer in model.named_modules():
            if isinstance(layer, (nn.Linear, nn.Conv2d)):
                self.hooks.append(layer.register_forward_hook(self._fwd(name)))
                self.hooks.append(layer.register_full_backward_hook(self._bwd(name)))

    def _fwd(self, name):
        def hook(module, inp, out):
            a = out.detach().cpu().flatten()
            self.act[name] = dict(
                mean=a.mean().item(),
                std=a.std().item(),
                dead=(a == 0).float().mean().item(),          # ReLU dead fraction
                hist=torch.histc(a, bins=self.bins).tolist())  # bin counts
        return hook

    def _bwd(self, name):
```

```
        def hook(module, gin, gout):
            g = gout[0]
            if g is not None:
                g = g.detach().cpu().flatten()
                self.grad[name] = dict(std=g.std().item(),
                                       hist=torch.histc(g, bins=self.bins).tolist())
        return hook

    def remove(self):
        for h in self.hooks: h.remove()

    def summary(self):
        for name, s in self.act.items():
            print(f"{name:12s}: mean={s['mean']:.3f}  std={s['std']:.3f}  "
                  f"dead_ratio={s['dead']:.3f}")

mon = ActivationMonitor()
model = nn.Sequential(nn.Linear(32,32), nn.ReLU(),
                      nn.Linear(32,32), nn.ReLU(), nn.Linear(32,1))
mon.register(model)
loss = model(torch.randn(64, 32)).sum()
loss.backward()
mon.summary()      # inspect PER LAYER -- never averaged across layers
mon.remove()
```

— end of document —